Requirements Analysis and Design (RAD), Sem01_2627

Project Proposal for SaigonBite

Online Food Delivery Hub Platform (OFDH)

Part 1 of the RAD Final Project

Team member	Student ID	Role	Owned main use cases
Nguyen Thanh Dat (Dat)	722I0004	Ordering, hub management and integration lead. Keeps the vision, backlog, risk list, Assumption Log and the business process diagram (Figure 1). Owns the Order and Hub state machines.	UC-01 Place an order; UC-03 Manage hubs and kitchen capacity
Le Minh Quan (Quan L.)	523K0049	Delivery and reporting. Keeps the schema, seed data and domain model. Owns the Delivery state machine.	UC-02 Deliver an order; UC-04 Produce management reports
Quan (Quan)	⟦ACTUAL m3_id: student ID of the third member⟧	Kitchen and catalogue; joins the build from Iteration 2. Owns the MenuItem state machine.	UC-05 Confirm or reject an incoming order; UC-06 Manage brands and menu items

Team: Team02

Date: 25 October 2026

File: Team02_Proposal_for_SaigonBite.pdf

Contents

1. System vision 2
1.1 The problem 2
1.2 Vision 3
1.3 Capabilities and expected benefits 3
1.4 Constraints fixed by the case 3
2. Approach and technology 4
2.1 Iterative Unified Process 4
2.2 Technology stack (team choice; lecturer asked to confirm Python, L4) 4
2.3 Three-layer design and the four roles 4
3. Overall business process 5
3.1 Figure 1 in five parts 6
4. Stakeholders, actors and use case ownership 11
4.1 Stakeholders and actors 11
4.2 Use case ownership 11
5. Scope 12
5.1 CRUD note 13
5.2 Integration demo scenario 13
6. Six-iteration roadmap 14
6.1 Gantt chart 15
6.2 Planned effort and buffer 15
7. Iteration 1 in detail 16
7.1 Day-by-day plan 16
7.2 Iteration 1 acceptance criteria (planned) 16
7.3 Plan vs actual 17
8. Prioritised backlog 17
9. Top 5 risks 18
10. Assumption Log and open items 19
10.1 Assumption Log (first entries) 19
10.2 Open items 20
11. Questions 21
11.1 Client questions 21
11.2 Questions for the lecturer 21
Appendix A. Generative AI Usage Log 22

1. System vision

1.1 The problem

SaigonBite runs cloud kitchens ("hubs") in Ho Chi Minh City. A hub has no dining room. It cooks only for delivery, for several virtual brands. SaigonBite has several hubs in different districts and plans more.

The whole process is manual today. Orders arrive by phone, by Zalo and through third-party delivery apps. Hub staff write each order on a paper ticket and pin it to a board, so nobody outside the kitchen can see how busy the hub is. At lunch and dinner one hub is overloaded and customers wait an hour, while a nearby hub is almost idle. When an ingredient runs out the apps are not updated, and staff phone customers to cancel or replace items. Shippers are called one by one or through a group chat. Most customers pay cash on delivery, and shippers hand the money to the hub at the end of the day, sometimes with mistakes. Foreign tourists struggle to order at all: they have no Vietnamese phone number, the menu is only in Vietnamese and their foreign cards are not accepted. Discount codes are announced in chat and applied by hand. The manager handles refunds for wrong or late orders one at a time. There are no reliable reports on what sells, which hub is busy or how long deliveries take.

1.2 Vision

SaigonBite will build one online platform on which customers order and pay, hub staff and shippers work each order through a clear sequence, the platform spreads demand across hubs, and managers read reports from the same records. This proposal covers the first release, which the team will analyse, design, build and test as one integrated application with six main use cases, two per student.

1.3 Capabilities and expected benefits

The benefits below are expected outcomes, not measured results. The system records timestamps from Iteration 3 so that they can be measured.

Pain point (from the case)	Capability	Expected benefit	Use case
Orders on paper tickets, phone and chat; nobody outside the kitchen sees the queue	One order record with a fixed list of states and a named person responsible for each state	Everyone sees the same order status; no re-typing between channels	UC-01, UC-05, UC-02
One hub overloaded while a nearby hub is idle	Active orders counted per hub; a hub at capacity stops taking orders and the customer is offered a quieter hub; a quiet-hub incentive (a configurable delivery-fee discount)	Fewer orders queued behind a full kitchen; demand moves to hubs with spare capacity	UC-01, UC-03
Menu items run out and customers are phoned afterwards	Only items available at the chosen hub can be ordered; if an item runs out later the kitchen rejects the order or replaces the item with the customer's consent	Fewer surprise cancellations	UC-01, UC-05, UC-06
Prices, fees and vouchers applied by hand; menus kept in several places	Full quote before confirmation: items, delivery fee based on distance, at most one voucher	The customer knows the price before paying; one menu per brand, kept by the admin	UC-01, UC-06
Tourists cannot register, read the menu or pay with foreign cards	Local and tourist customer types with different online payment options; both can pay cash	Tourists get a payment route that exists	UC-01
Shippers reached one by one; cash handed over at the end of the day with mistakes	The shipper sees ready orders, records pickup, delivery, cash collected or failure per order (built). Office staff reconcile the handed-over cash against those records (in the process, not built)	Cash is tied to a specific order; the end-of-shift handover has a record to check against	UC-02
No reliable reports	Sales, hub workload and delivery performance computed from the shared records	Managers compare hubs using the same definitions	UC-04

Complaints, refund decisions, refund payouts and customer ratings stay in the business process (Section 3) but are not built in this release (Section 5). An online payment that must be refunded is stored as REFUND_PENDING. Menu translation for tourists is also outside this release.

1.4 Constraints fixed by the case

- Hubs cook only for delivery, have limited kitchen capacity and work with several brands.
- A customer is either a local customer or a foreign tourist. Registration details and online payment options differ. Both can pay cash on delivery.
- An order passes through a sequence of states from placement until it is delivered, cancelled or fails.
- Deliveries are limited to a maximum distance from the hub, and the delivery fee depends on distance.
- A customer can use at most one voucher per order.
- A busy-kitchen rule and a quiet-kitchen incentive exist. The figures will come from the client.
- Customer addresses and phone numbers are private.

2. Approach and technology

2.1 Iterative Unified Process

The team follows iterative Unified Process practices in six time-boxed iterations. Each iteration analyses, designs, builds and tests a working increment, and ends with a joint review by the whole team. Inside each iteration the work follows the six core processes of the textbook (chapter 1, RMO example): identify the problem and obtain approval, plan and monitor the project, discover and understand the details, design the system components, build, test and integrate, and complete system tests and deploy.

Why it suits this project:

- The case is incomplete on purpose. Capacity limits, fees, payment rules and report definitions come from the client Q&A. Short iterations let the team start on logged assumptions and change course when the answers arrive.
- The six use cases share one Order record. Building a thin working slice in Iteration 1 and growing it exposes integration problems early, when they are cheap.
- The riskiest rules are tackled first: the capacity check in Iteration 2 and the full payment and delivery paths in Iteration 3.
- The deadlines are fixed (Part 1, Part 2 and Part 3 presentations) and the team is small. Fixed-length iterations with an exit check make progress visible. We keep the iteration, the backlog and the joint review, which is enough process for three people.

2.2 Technology stack (team choice; lecturer asked to confirm Python, L4)

Part	Choice	Why
Language	Python 3.13 (3.12 or later works)	The whole team is strongest in Python.
Web framework	Flask with Jinja2 templates	Small, routes stay thin, templates map directly to screens.
Data access	sqlite3 data-access objects (DAO) with hand-written SQL, no ORM	The report asks for the database mapping and the SQL; hand-written SQL keeps both in plain view.
Database	SQLite	No server to install; schema and seed data load with one script.
Testing	pytest	Test names can carry test-case IDs; output serves as the recorded result.
Login	Flask-Login with seeded users	Role checks are enforced on the server, not only by hiding buttons.

The system is one application with one database.

2.3 Three-layer design and the four roles

Layer	Role	Python package	Examples
Presentation	View	`presentation/`	One Flask blueprint per use case; Jinja2 templates
Business	Controller	`business/controllers/`	`PlaceOrderController`, `HubController`, `DeliveryController`, `ReportController`, `KitchenController`, `MenuController`; one method per system operation in the SSD
Business	Domain	`business/domain/`	`Order`, `OrderLine`, `Hub`, `Customer` with `LocalCustomer` and `TouristCustomer`, `Payment`, `Delivery`, `Brand`, `MenuItem`
Data access	DAO	`data_access/`	`OrderDAO`, `HubDAO` and others, with SQL written by hand

Deployment is `python -m venv .venv`, `pip install -r requirements.txt`, `python init_db.py`, `flask run`, or `docker compose up --build` for the same app in a container.

3. Overall business process

A customer browses hubs and menus, picks a hub and items, and receives a quote with the delivery fee and voucher discount. If the hub is busy, the platform suggests a quieter hub and the customer chooses again. The customer pays cash on delivery (COD) or through the online stub; if online payment fails, the customer retries or switches to COD. The platform then creates the order as PLACED. While the order is PLACED or CONFIRMED, that is before cooking starts, the customer may cancel; cancellation is a separate small activity (Figure 1e, Cancel order), analysed but not built. The kitchen confirms or rejects the order. If an item runs out after ordering, the kitchen offers a replacement and the customer decides; if the customer declines, the order is rejected. A rejected order voids a COD payment, and an online payment is owed back in full (REFUND_PENDING) without a decision. The kitchen prepares the order and marks it READY, and a shipper picks it up and delivers it. On success the shipper records the COD collection and the order becomes DELIVERED. On failure the shipper reports it and returns the food; the order becomes DELIVERY_FAILED, a COD payment is voided and an online payment is again owed back in full (REFUND_PENDING) without a decision. At the end of the shift the shipper hands the cash to the hub and office staff reconcile it. After delivery the customer receives a receipt and may rate the order, or complain about a wrong or late order. A complaint is the only path that needs a person to decide: office staff approve a refund (online or in cash), or deny it and tell the customer why, and the payment stays PAID. Managers read reports in a separate small activity from the same shared records. Grey actions are in the process but not built in this release: rating, complaint, the refund decision and paying out any refund owed are deferred core features (A-09); the build records a refund owed as REFUND_PENDING, and the Cancel order activity is analysed only, like the other supporting use cases (A-08). A flow-final symbol ends one path; an activity-final symbol ends the whole activity it belongs to.

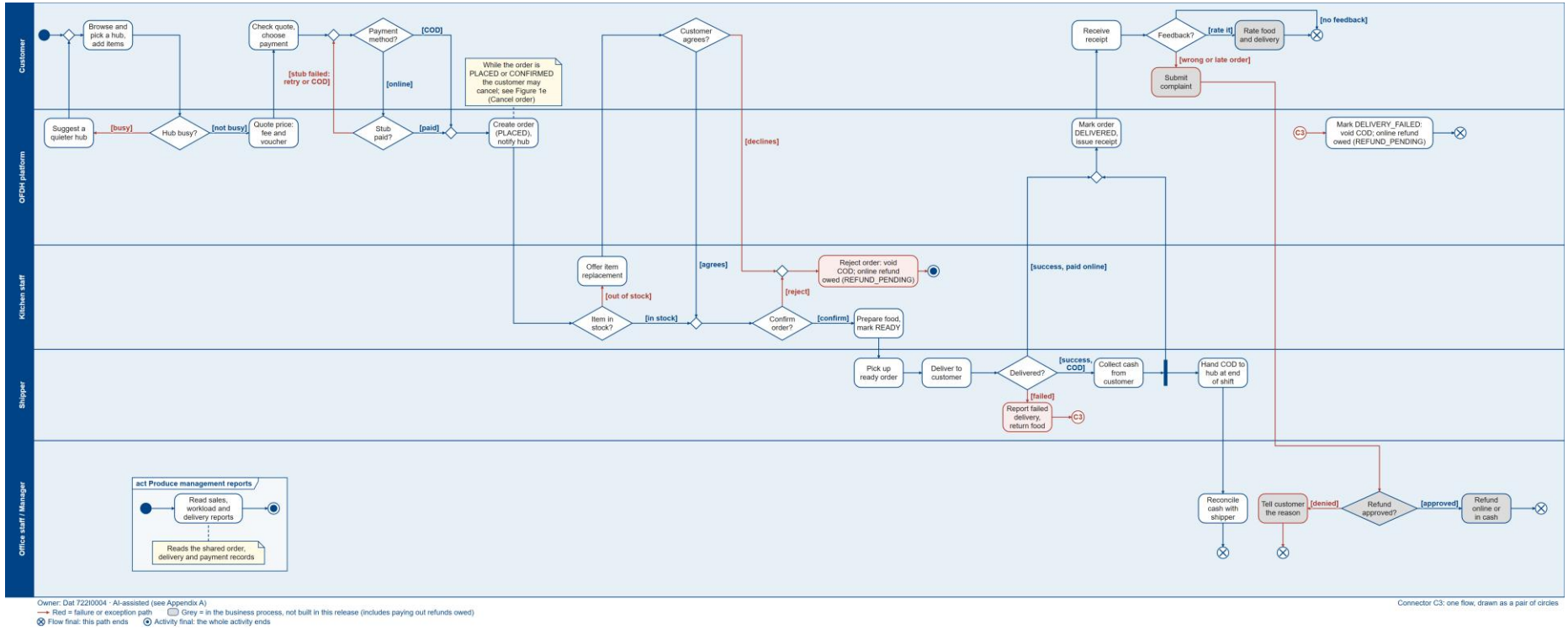


Figure 1 Overall business process (owner: Dat 722I0004; AI-assisted, see Appendix A)

3.1 Figure 1 in five parts

Figures 1a to 1e show the same process in five enlarged parts, one per page, so that the text can be read in print. Parts A to D follow the order process, and the connector codes show where the flow continues in the next part. Part E is the separate Cancel order activity, which is analysed only (A-08).

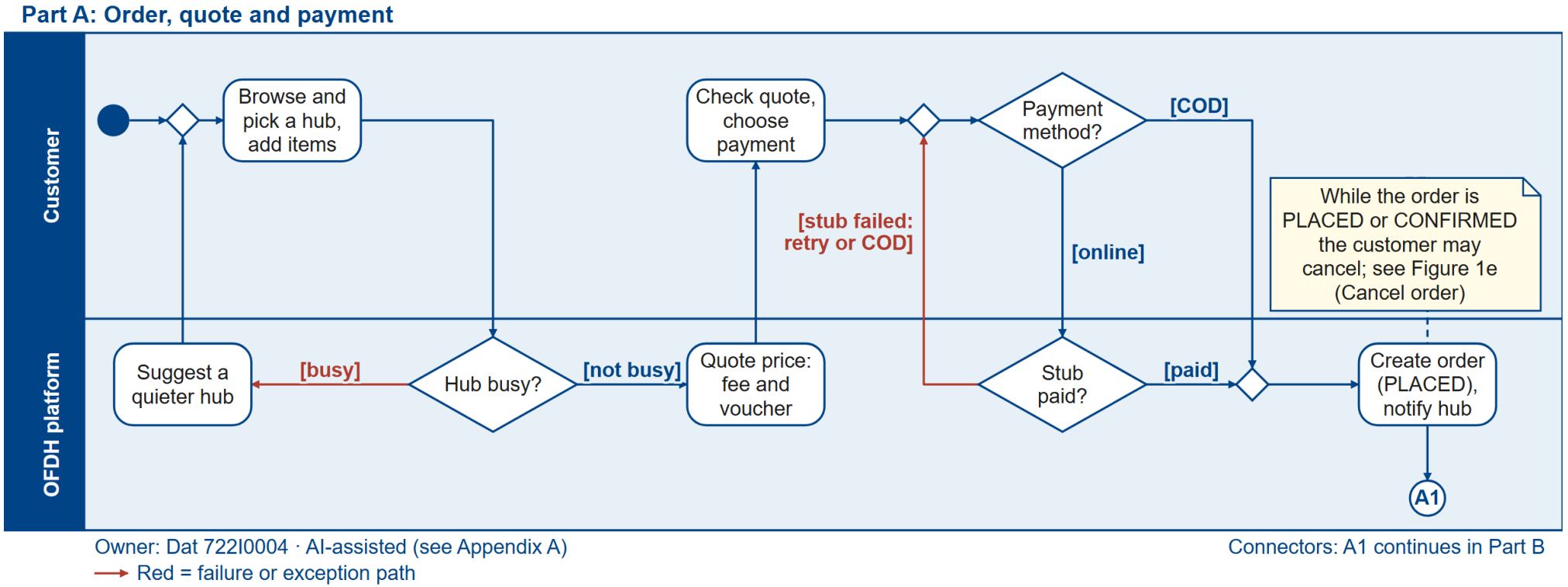


Figure 1a Overall business process, part A: Order, quote and payment (owner: Dat 722I0004; AI-assisted, see Appendix A)

Part B: Kitchen and item replacement

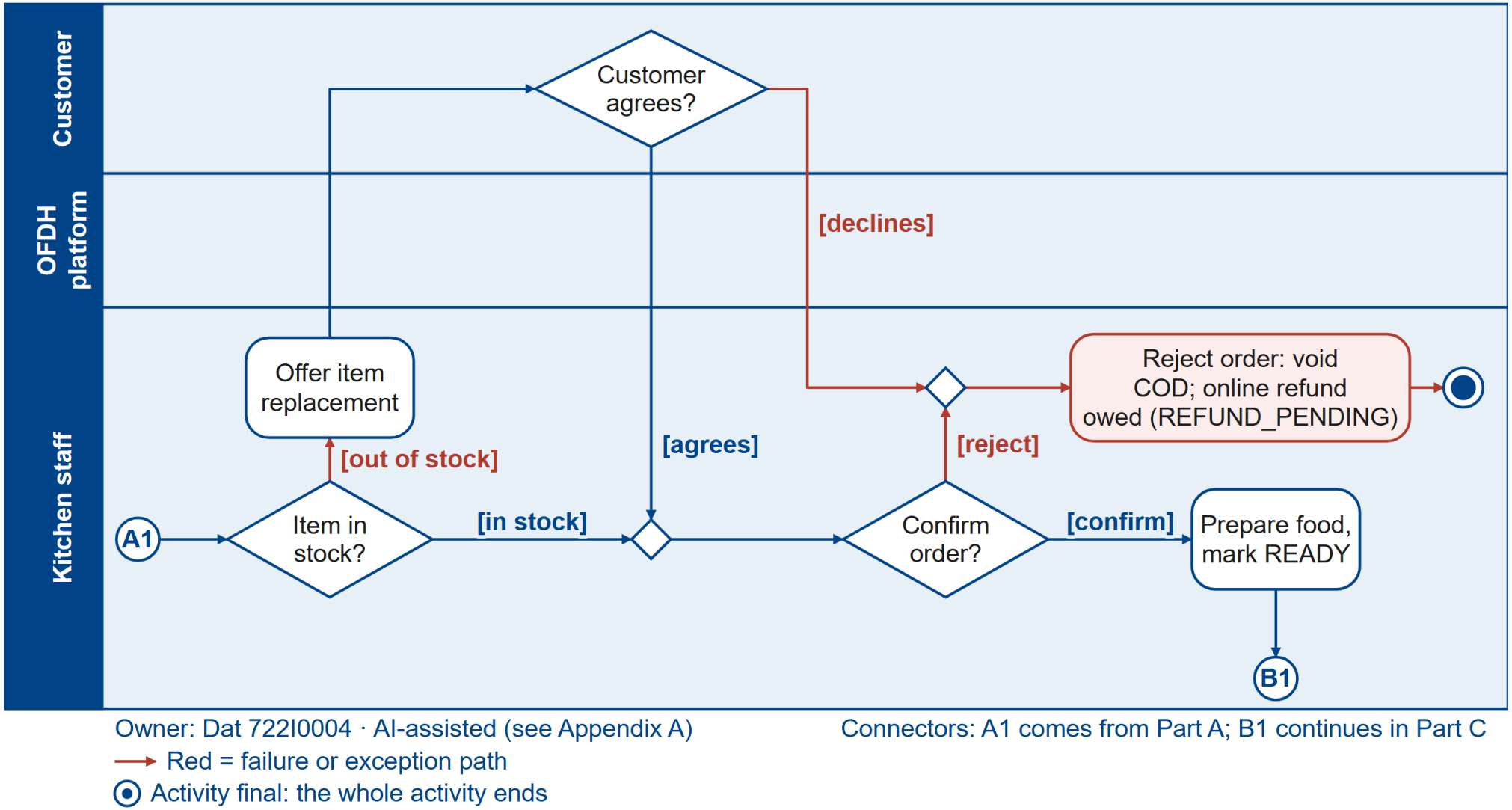
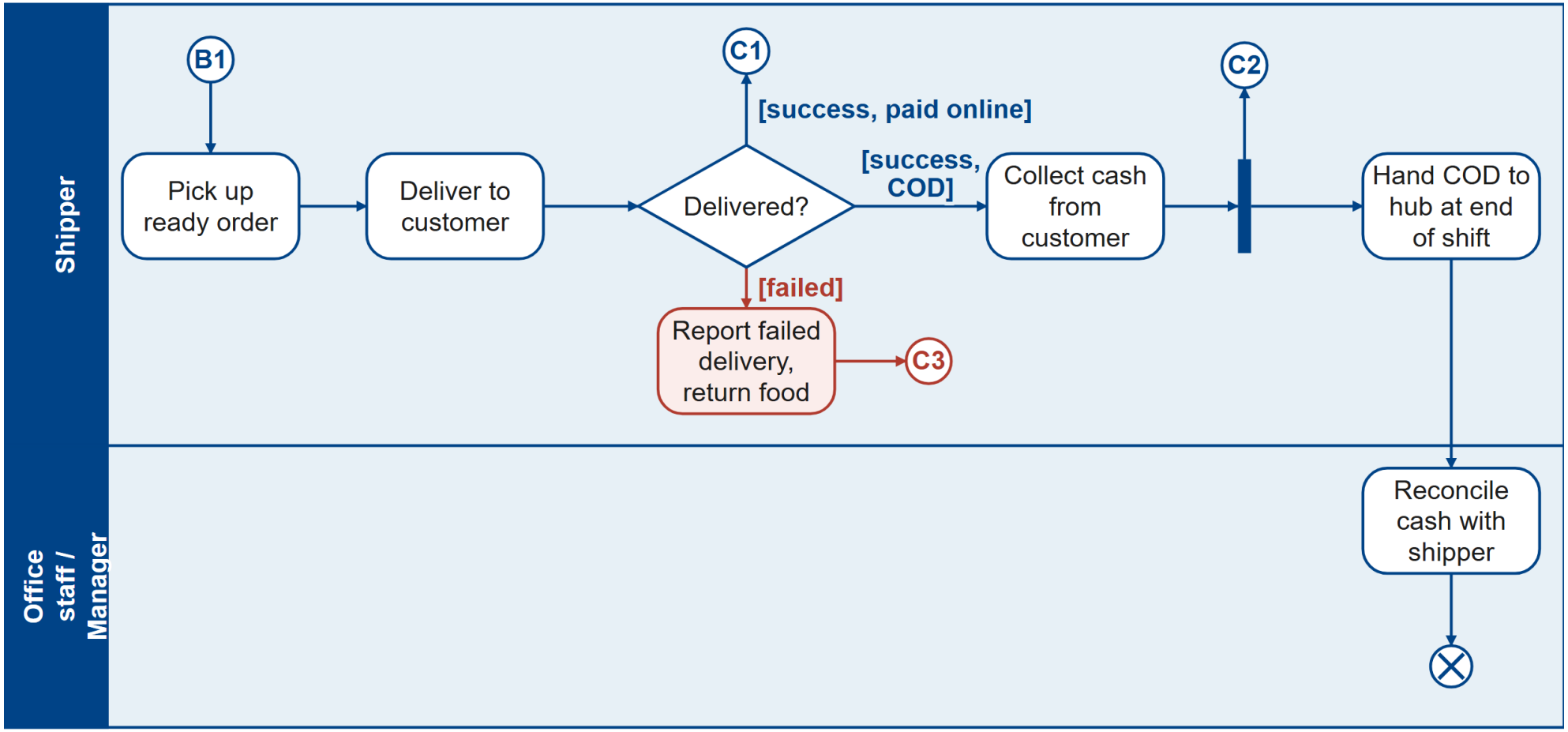


Figure 1b Overall business process, part B: Kitchen and item replacement (owner: Dat 722I0004; AI-assisted, see Appendix A)

Part C: Delivery and cash



Owner: Dat 722I0004 · AI-assisted (see Appendix A)

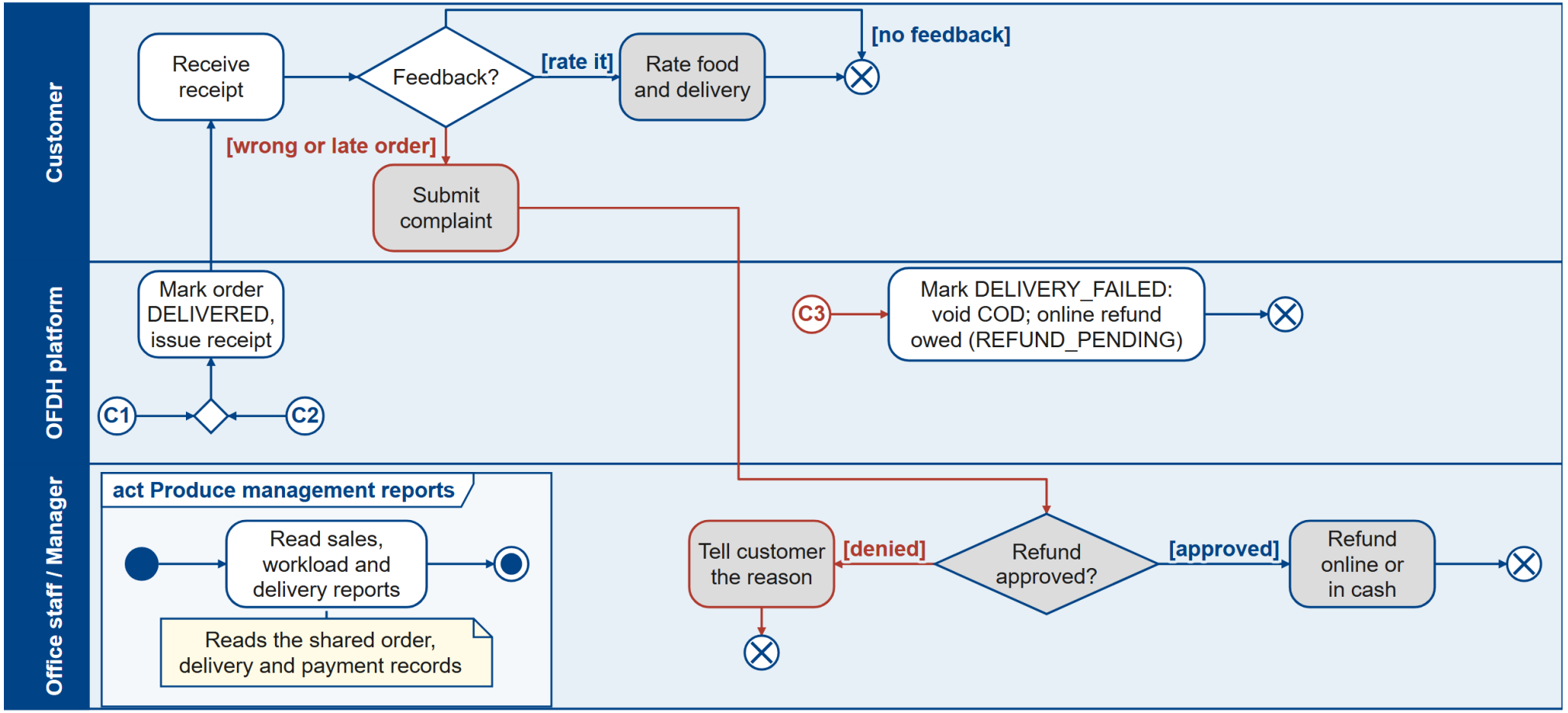
→ Red = failure or exception path

⊗ Flow final: this path ends

Connectors: B1 comes from Part B; C1, C2, C3 continue in Part D

Figure 1c Overall business process, part C: Delivery and cash (owner: Dat 722I0004; AI-assisted, see Appendix A)

Part D: Receipt, failed delivery, refund and reports



Owner: Dat 722I0004 · AI-assisted (see Appendix A) Connectors: C1, C2, C3 come from Part C

→ Red = failure or exception path ▭ Grey = in the business process, not built in this release (includes paying out refunds owed)

⊗ Flow final: this path ends ◉ Activity final: the whole activity ends

Figure 1d Overall business process, part D: Receipt, failed delivery, refund and reports (owner: Dat 722I0004; AI-assisted, see Appendix A)

Part E: Cancel order (separate activity, analysed only)

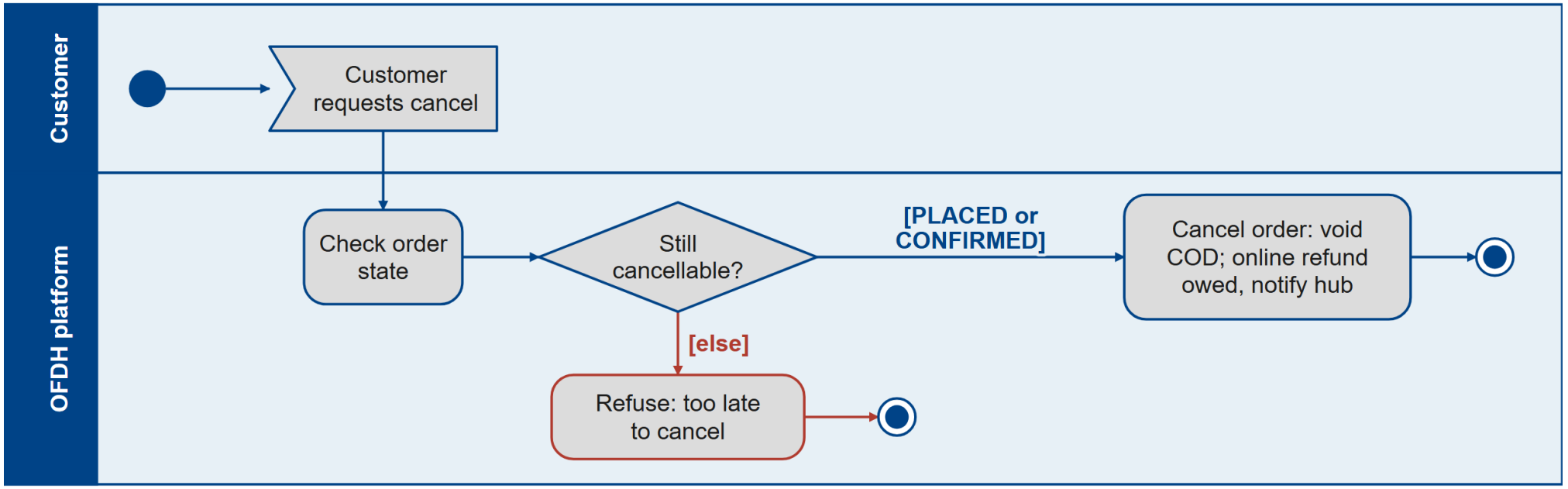


Figure 1e Overall business process, part E: Cancel order, a separate activity analysed only (A-08) (owner: Dat 722I0004; AI-assisted, see Appendix A)

4. Stakeholders, actors and use case ownership

4.1 Stakeholders and actors

Stakeholder / actor	Goal	What the system must give them
Customer (local or tourist)	Order food and know the price, pay, receive the order	Quote before confirmation; payment options matching customer type; busy-hub message with a quieter hub
Kitchen staff	Cook only what the hub can handle	Incoming orders to confirm, reject or change with the customer's consent (UC-05); preparing and ready states
Shipper	Deliver, and account correctly for cash	Ready orders assigned to them; delivered, cash collected, or failed; a record to hand the cash over against
Admin	Keep hubs, capacity, brands and menus correct	Create, view, update and archive hubs (UC-03), brands and menu items (UC-06)
Manager	Decide which hubs and brands need attention	Sales, hub workload and delivery performance reports
Office staff	Settle complaints fairly and check the cash	Per-order cash records to reconcile against the shipper's handover; complaint and refund decision (in the process, not built)
Payment provider (outside the system; simulated by our stub)	Return an online payment result	A team-built stub that returns success or failure, so both branches can be tested
SaigonBite management (client)	A platform that solves the stated problems and can grow	Answers to the client questions in Section 11

4.2 Use case ownership

Each student owns exactly two main use cases, so a team of three has six. Together they cover groups (a) ordering, (b) fulfilment, (c) full CRUD and (d) report.

ID	Use case	Candidate #	Actor	Group	Owner	State machine
UC-01	Place an order (price it, apply a voucher, choose payment)	#3	Customer	(a) ordering	Dat 722I0004	Order
UC-02	Deliver an order (pick up, deliver, confirm cash or report failure)	#10	Shipper	(b) fulfilment	Quan L. 523K0049	Delivery
UC-03	Manage hubs and kitchen capacity	#13	Admin	(c) full CRUD	Dat 722I0004	Hub
UC-04	Produce management reports (sales, hub workload, delivery performance)	#16	Manager	(d) report	Quan L. 523K0049	none (read-only)
UC-05	Confirm or reject an incoming order (including an item replacement the customer agrees to)	#7	Kitchen staff	(b) fulfilment	Quan ⟦ACTUAL m3_id: student ID⟧	uses Order
UC-06	Manage brands and menu items	#14	Admin	(c) full CRUD	Quan ⟦ACTUAL m3_id: student ID⟧	MenuItem

The owner is the author of every model of the use case, with the student ID on each diagram, and writes most of its code. Another member reviews it, and commits are recorded under the person who did the work. The third member joined after Iteration 1; Dat pairs with the new member in Iteration 2 so the shared code is familiar from the first week. The kitchen's preparing and ready steps and the login with seeded users are minimum supporting builds. Dat is the integration lead, who merges the branches, owns the integration demo scenario (Section 5.2) and runs it at the end of I3 and I4. Quan L. keeps the shared schema and seed data so that every use case reads the same records.

Part 3 plan: Dat presents UC-01, Quan L. presents UC-02 and Quan presents UC-05.

5. Scope

Level	Items	What the team does
Planned build for Part 2 (main use cases)	UC-01 to UC-06	Analyse, design, build and test in one application. Built so far (Iteration 1): a subset of UC-01, one persisted COD order
Minimum support	Kitchen preparing and ready steps; role-based login with seeded users	Build the minimum so one order can travel from placement to delivery
Analysed, not built	Register customer and addresses; manage vouchers; manage menu availability; manage user accounts; track order; cancel or edit order (Cancel order activity, Figure 1e); archive old orders	Use case list, description and CRUD mapping; data comes from seed files. Depth depends on the lecturer's answer (L1)
Deferred core features	Handle complaint and refund (the decision and the payout); rate an order	Kept in the business process (grey actions in Figure 1), use case list and backlog; not built in this release (A-09, L3)
Future (from the case)	Loyalty points, subscription meal plans, group orders, scheduled orders	Mentioned in the vision and conclusions only

Order states are fixed: PLACED, CONFIRMED, REJECTED, PREPARING, READY, OUT_FOR_DELIVERY, DELIVERED, DELIVERY_FAILED, CANCELLED. The customer may cancel from PLACED or CONFIRMED, not once the order is PREPARING. An order being DELIVERED is not the same as the payment being PAID. The Delivery has its own states in Quan L.'s state machine; when the delivery is PICKED_UP, the order moves to OUT_FOR_DELIVERY.

Payment states are listed here once and used the same way in the process, A-06, Q3 and the slides:

Payment state	Meaning
PENDING	A COD payment waits for the cash to be collected
PAID	Online payment succeeded, or the shipper confirmed that the cash was collected
FAILED	The stub returned failure for one online attempt; no order is created, so this is shown to the customer and logged only
VOIDED	A COD payment that will never be collected (kitchen rejection, cancellation or failed delivery); no money moved
REFUND_PENDING	An online payment that is owed back in full after a rejection, cancellation or failed delivery. No decision is needed; the payout itself is not built in this release (A-09)
REFUNDED	The money was paid back: an automatic refund above, or a complaint refund approved by office staff (analysed, not built)

Refund policy (proposed default, A-06 and Q3): rejection, cancellation and failed delivery are the platform's or the kitchen's outcome, so an online payment is owed back in full without a decision. The build records this as REFUND_PENDING; paying the money out (REFUNDED) is deferred. A complaint about a delivered order is the only case that needs office staff to decide. If they deny it, the payment stays PAID and the customer is told the reason.

5.1 CRUD note

After CRUD verification every persistent class must be created, read, updated and deleted (or archived) by some use case. The team's mapping:

Class	Covered by	Implementation
Hub	UC-03: create, read, update, archive (guard: no active orders)	Planned, I2 (read only today, from seed data)
Brand, MenuItem, HubMenuItem	UC-06: create, read, update, archive (an archived item can no longer be ordered; old orders keep their copied name and price)	Planned, I3 (read only today, from seed data)
Order, OrderLine	Create: UC-01. Read and update: UC-05, kitchen steps, UC-02. Archive: the supporting use case "archive old orders"	Create and read built in I1; update planned (I2, I3); archive analysed only
Payment, Delivery	Create and update: UC-01, UC-05 and UC-02. Read: UC-04. Archive follows the Order	COD payment created in I1; the rest planned (I2 to I4); archive analysed only
Customer and Address, Voucher, UserAccount and Shipper	Supporting use cases analysed in the list above (applying a voucher at checkout is built in UC-01; managing vouchers is not)	Seed data, unless the lecturer asks otherwise (L1)

Design for the analysed archive use case: when it is built, an archived order carries an `archived_at` timestamp. It disappears from operational screens, reports still read it, and nothing is physically deleted. Payment and Delivery are filtered through the Order's flag. Writing this mapping into the Assumption Log does not make it approved: the team asks the lecturer (L1) and records the answer.

Two scenarios, decided by Sat 31/10 (end of I2):

Scenario	What changes	Who builds it, from whose spare hours	Spare hours left
L1 accepted: analysis plus seed data is enough	Nothing	n/a	Each member keeps 3 hours in I3 and 3 in I4; I5 keeps 12 (buffer day 13/11)
L1 rejected for named classes	UC-06 builds a generic admin pattern (list, create, edit, archive) in I3. Each named class reuses it, about 6 hours including a review by another member: Voucher first, then the next class the lecturer names	Voucher: Quan, 3 hours in I3 and 3 in I4. Second class: Quan L., 3 hours in I3 and 3 in I4. Third class: Dat, the same way	Two classes: Dat keeps 6 spare hours for integration, I5 keeps 12. A third class uses every spare hour of I3 and I4; the team then tells the lecturer that nothing else fits before Part 2

5.2 Integration demo scenario

1. The Admin lowers the capacity of a seeded hub that already has a menu (the District 1 hub) (UC-03) and marks one menu item as archived (UC-06).
2. A customer places orders until the hub is busy, is blocked and is offered a quieter hub (UC-01). The archived item cannot be ordered.
3. Kitchen staff confirm one order, reject another (the online payment becomes REFUND_PENDING) and move the confirmed orders to preparing and ready (UC-05).
4. A shipper delivers one COD order and reports one failed delivery (UC-02).
5. The Manager reads the reports; the figures match a table the team calculated by hand beforehand (UC-04).

Create Hub and Archive Hub are tested separately. A new hub has no menu, so it cannot take orders in the demo.

6. Six-iteration roadmap

Assumed effort is about 4 focused hours per student per day. Iteration 1 had two people (48 person-hours); from Iteration 2 the team has three. The lecturer questions are asked at the Part 1 presentation and the team needs the answers by the end of I2. Nothing planned for Part 2 relies on I6: Iteration 5 holds its own buffer day.

Iteration and dates	Goal and increment	Use cases and stories	Deliverables	Owners	Exit criterion
I1 17/10 - 22/10 (6 days)	A persisted COD order on a runnable skeleton	UC-01 subset (one hub, COD, flat fee); US-01, US-12 (login seed)	Vision, backlog, UC-01 subset description, activity diagram, SSD, Order state machine v1; schema, seed, DAO; checkout slice; pytest results; run instructions	Dat: vision, UC-01 subset analysis and checkout slice. Quan L.: events and actors, domain and schema, DAO, seed, UC-02 draft	Acceptance criteria AC-1 to AC-9 (Section 7.2)
Part 1 packaging 23/10 - 25/10	Proposal, slides, rehearsal		Proposal PDF, slides, two timed rehearsals with all three presenters	All three	Proposal submitted 25/10; presented live 26/10
I2 27/10 - 31/10	Hub CRUD, capacity rule, kitchen board	UC-03; busy-hub check in UC-01; UC-05; UC-02 and UC-06 analysis; US-03, US-04, US-05, US-13, US-21	UC-03 and UC-05 artefact sets and tests; Hub state machine; UC-02 analysis and Delivery state machine; UC-06 analysis	Dat: UC-03, workload check, pairing with the new member. Quan L.: UC-02 analysis, kitchen preparing and ready steps. Quan: UC-05, UC-06 analysis	Boundary test passes (capacity minus 1 accepted, capacity blocked, two simultaneous orders do not both pass); an order moves PLACED to READY; tag `v0.2`
I3 01/11 - 05/11	Full ordering, delivery and menu management	UC-01 complete; UC-02; UC-06; US-01 (full), US-02, US-06, US-07, US-08, US-14, US-15, US-16, US-22	Voucher, distance fee, quiet incentive, payment options and stub, tourist customer; UC-02 design, code, tests; UC-06 code and tests; timestamps	Dat: UC-01 rules and payment stub. Quan L.: UC-02. Quan: UC-06 and the item replacement in UC-05	One order ID travels from placement through delivery; one failed delivery and one rejection handled; tag `v0.3`
I4 06/11 - 10/11	Reports, integrated demo, Figma	UC-04; US-09, US-10, US-11; all six integrated	UC-04 artefacts with expected values calculated by hand first; Figma for each student's use cases	Quan L.: UC-04. Dat: integration lead, missing exceptions. Quan: UC-05 and UC-06 tests and Figma	Demo scenario (5.2) runs end to end; report output equals the hand-calculated table; tag `v0.4`
I5 11/11 - 15/11	Final models, system tests, deployment, Part 2 report. Feature freeze 12/11; buffer day 13/11	All six; `v0.9-part2`	Final design models, database mapping and SQL, test plan with recorded results, deployment instructions, Contribution Statement signed by all three, AI usage log, Part 2 report PDF	All three; Dat leads integration	Code frozen 12/11; fresh-machine run passes; report submitted 15/11
I6 17/11 - 29/11	Fix Part 2 feedback, release `v1.0-final`, Part 3	All six	Corrections to Part 2 feedback, `v1.0-final`, Part 3 slides and video, updated Contribution Statement and AI log	All three	Release tagged; Part 3 package submitted 29/11

6.1 Gantt chart

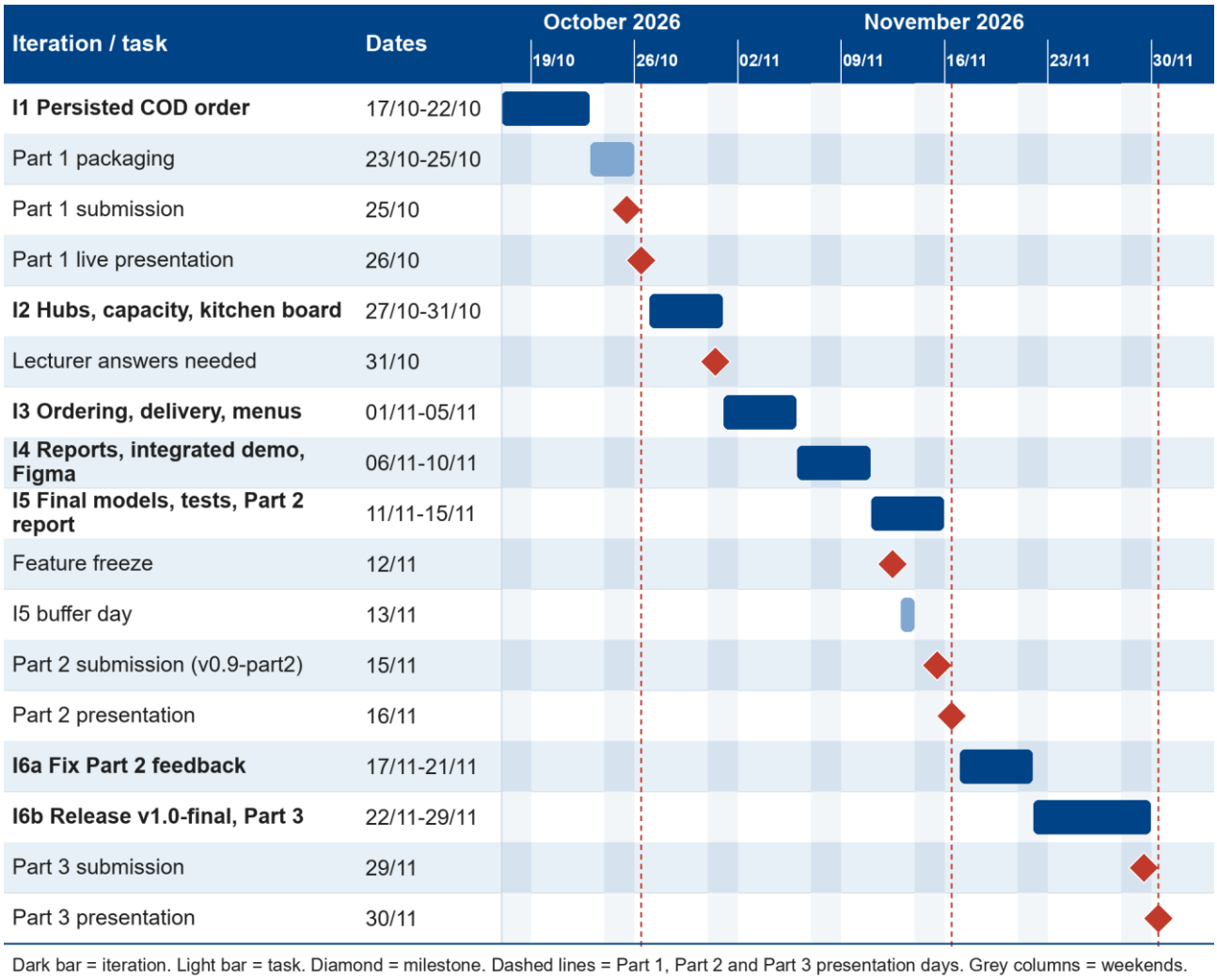


Figure 2 Roadmap from 17/10 to 30/11/2026 with the Part 1 (26/10), Part 2 (16/11) and Part 3 (30/11) presentation days

I6a and I6b together form I6 (17/11 to 29/11).

6.2 Planned effort and buffer

Baseline: 4 focused hours per student per day. Build hours are code, automated tests, seed and deployment scripts; the rest goes to analysis, design, documents and cross-review. The buffer is reserved time that is not planned to any story. The I1 row was planned for two people; the third member's I2 hours include about 4 hours of onboarding.

Iteration	Days	Capacity (person-hours)	Dat planned	Quan L. planned	Quan planned	Buffer	Build hours Dat / Quan L. / Quan
I1	6	48	24	24	-	0	12 / 8 / -
Part 1 packaging	3	36	10	10	10	6	0 / 0 / 0
I2	5	60	18	18	18	6	10 / 6 / 8
I3	5	60	17	17	17	9	12 / 8 / 8
I4	5	60	17	17	17	9	8 / 5 / 6
I5	5	60	16	16	16	12 (13/11)	5 / 3 / 3
I6	13	156	44	44	44	24	12 / 7 / 7
Total	42	480	146	146	122	66	59 / 37 / 32

Dat carries the most build hours because Dat builds UC-01, UC-03 and the integration. Each member plans 17 of 20 hours in I3 and I4, so each keeps 3 spare hours per iteration; those 18 person-hours cover the CRUD scenario

in 5.1 (Section 5.1 says whose hours) and integration surprises. The feature freeze on 12/11 stops new features; the I5 buffer day on 13/11 is for fixing and re-testing defects found after the freeze, or for covering a missing member. Should and Could items (US-17, US-20) are worked only if a buffer is still unused at the end of its iteration. The hours of I2 to I6 are re-checked at the end of each iteration.

7. Iteration 1 in detail

7.1 Day-by-day plan

Iteration 1 ran from 17/10 to 22/10 with the two original members, about 4 focused hours each per day. The days follow the six core processes of textbook chapter 1.

Day	Textbook core process	Dat (UC-01)	Quan L. (shared foundation and UC-02)	End-of-day evidence
D1	CP1 identify the problem and obtain approval; CP2 plan and monitor the project	Write the vision from the case; problem-to-capability table; UC-01 user stories; MoSCoW backlog v1; risk list v1	Repository and Flask skeleton with the four packages; `init_db.py`; events and actors v1; use case diagram v1	Backlog v1; skeleton runs with `flask run`
D2	CP3 discover and understand the details	UC-01 subset description (COD, one hub): pre- and post-conditions with IDs, quote rules; five client questions; business process diagram (Figure 1) v1	Domain model v1; CRUD inventory v1	Class and order state names fixed; UC-01 description complete
D3	CP3 and CP4 design the system components	UC-01 activity diagram, SSD, Order state machine v1, checkout sketch	`schema.sql` and `seed.sql`; UC-02 description draft; check the SSD against the schema	SSD operations, schema columns and state names agree
D4	CP4 and CP5 build, test and integrate	`PlaceOrderController` and `Order` domain code	DAO base; Hub and menu reads; `OrderDAO` insert	A COD order is saved by a test
D5	CP5 build, test and integrate	Checkout page; pytest cases for UC-01 (success, invalid input, repeated submit)	Role checks on the server; README run steps	pytest run recorded with date and commit
D6	CP6 complete system tests and deploy	Fix defects; run UC-01 acceptance scenarios; update risks and assumptions	Run the README on a second machine; revise the roadmap using the I1 hours	Second-machine result; I1 review; plan vs actual (7.3)

7.2 Iteration 1 acceptance criteria (planned)

ID	Criterion
AC-1	From a clean checkout on a second machine, the setup commands run without manual edits and the home page opens.
AC-2	`init_db.py` loads at least two hubs with brands and menu items, one local customer with an address, and one user for each of the roles customer, kitchen staff, shipper, admin and manager.
AC-3	A logged-in customer selects a hub, items, quantities and an address, and sees item subtotal, delivery fee and total before confirming. The fee is a flat seeded value until the client answers the fee question.
AC-4	Confirming creates one Order in state PLACED with its lines and a COD Payment in a single transaction. The confirmation shows the order ID and total.
AC-5	Reading that order back through a new database connection returns the same lines and the same total.
AC-6	An empty cart, quantity of zero or less, an item not available at the hub, and a missing address each produce a clear message and no new order.
AC-7	Pressing confirm twice creates exactly one order.
AC-8	A user who is not logged in is redirected to login, and a user without the customer role cannot place an order, checked on the server.
AC-9	pytest covers AC-4 to AC-8 with results recorded. The UC-01 description, activity diagram, SSD, Order state machine and schema use the same names.

7.3 Plan vs actual

Actual values come from the repository history (https://github.com/Dat071104/SaigonBite) and the pytest output. The Iteration 1 code skeleton, schema and seed, UC-01 subset and pytest suite were generated with AI assistance (Claude) on D3; the team then reviewed that code, found two defects the tests had missed and fixed them (Appendix A). The table states for each day what was finished and how.

Day	Planned evidence	Actual	Variance and action
D1	Backlog v1; skeleton runs	Planning only: plan v1 and v2 (15/10, AI-assisted); vision, backlog and risks in plan v3. No repository yet.	Skeleton moved to D3 and built with AI-assisted scaffolding; no effect on the end date.
D2	Names fixed; UC-01 description	Client questions and CRUD table in plan v3; UC-01 description, domain model and CRUD inventory drafted with AI on D3.	D2 analysis finished on D3 instead; names fixed in code, schema and docs at the same time.
D3	SSD, schema and states agree	AI-assisted session on 19/10: skeleton, schema and seed, UC-01 subset, 81 pytest tests, five analysis documents.	D4 and D5 scope pulled into D3. The UC-01 activity diagram followed on 20/10.
D4	COD order saved	Done early on D3 (commits 622549a, 2b11e42, 6595553): domain, DAOs, controller; a COD order is saved by the tests.	Ahead of plan; the time was used for the review on 20/10.
D5	pytest run recorded	Done early on D3 (049673d, tag i1-evidence): views, README, 81 tests, all passed.	Ahead of plan; checked again in the review on 20/10.
D6	Second-machine run; review	Review on 20/10 found two defects the 81 tests missed (price change after the quote, archived hub missing from history); both fixed with tests (a6ccbee). Fresh clone in a clean Docker container: 90 passed (40ef14e). A second review closed a request that skipped the price check (e93336a, 93 passed on a fresh clone). A team member ran the app and the 90 tests on a Mac with Docker.	None: the second-machine run and the review were done ahead of D6.

Measure	Plan	Actual
Person-hours	48	Not logged per day. Most of the build was one AI-assisted session on 19/10, then the review on 20/10. From I2 the team logs hours per iteration.
Acceptance criteria met	9 of 9	9 of 9
pytest result	AC-4 to AC-8 covered	93 passed, 0 failed on a fresh clone after the round-2 review fix (commit e93336a); 90 passed on a team member's Mac; 81 passed at tag i1-evidence (19/10)
Clean checkout and second machine	Passes	Passes: a team member ran the setup, the app and the 90 tests on a Mac with Docker, no manual edits. Also a fresh clone in a Linux container (Python 3.13.16).
Change to the roadmap	None expected	A third member joins from I2, so the team owns six main use cases (UC-05, UC-06) and capacity rises to 60 person-hours per iteration.
Lesson that changed the plan	None expected	Green tests are not proof: review found a bug 81 tests missed. Every finding now gets a test.

8. Prioritised backlog

MoSCoW priority. A story is Must when the main use case cannot work without it, or when the case states it as a fixed rule (voucher, payment options by customer type, busy and quiet kitchen). Should and Could items are worked only from unused buffer time (Section 6.2).

ID	User story	Priority	Iteration	Owner
US-01	As a customer, I want to see the full quote and confirm a COD order so that I know what I will pay.	Must	I1, completed in I3	Dat
US-02	As a customer, I want delivery refused beyond the maximum distance and the fee based on distance.	Must	I3	Dat
US-03	As a customer, I want a busy hub to refuse my order and suggest a quieter hub, so that I do not wait an hour.	Must	I2	Dat

ID	User story	Priority	Iteration	Owner
US-04	As an admin, I want to create, view and update a hub and its capacity.	Must	I2	Dat
US-05	As an admin, I want to archive a hub that has no active orders so that history is kept.	Must	I2	Dat
US-06	As a shipper, I want to pick up a ready order assigned to me.	Must	I3	Quan L.
US-07	As a shipper, I want to confirm delivery and cash collected.	Must	I3	Quan L.
US-08	As a shipper, I want to report a failed delivery with a reason.	Must	I3	Quan L.
US-09	As a manager, I want a sales report with a date filter.	Must	I4	Quan L.
US-10	As a manager, I want a hub workload report.	Must	I4	Quan L.
US-11	As a manager, I want a delivery performance report.	Must	I4	Quan L.
US-12	As a user, I want access limited by role and by the records assigned to me.	Must	I1 to I5	Quan L.
US-13	As kitchen staff, I want to mark a confirmed order as preparing and then ready.	Must	I2	Quan L.
US-21	As kitchen staff, I want to confirm or reject an incoming order, or offer a replacement the customer can accept, so that I only cook what I can deliver.	Must	I2, replacement in I3	Quan
US-22	As an admin, I want to create, update and archive brands and menu items and set which hubs carry them.	Must	I3	Quan
US-14	As a customer, I want to apply one voucher and see the discount.	Must	I3	Dat
US-15	As a customer of a quiet hub, I want a fee discount so that I choose it. Minimal mechanism: one configurable incentive, a delivery-fee discount for a hub under the quiet threshold; threshold and amount sit in configuration until Q1 is answered.	Must	I3	Dat
US-20	As the business, I want advanced quiet-hub incentive variants (time windows, stacking with a voucher).	Should	I4 or I5 buffer, only if time allows	Dat
US-16	As a local customer or tourist, I want online payment options for my customer type through the stub, and to retry or choose COD if it fails.	Must	I3	Dat
US-17	As a customer, I want to follow my order's status.	Could	I4, only if ahead of plan (buffer time)	Dat
US-18	Handle complaint and refund; rate an order; register customer and addresses; manage vouchers, menu availability and user accounts; cancel or edit order; archive old orders.	Won't (this release; stays in process and use case list)	n/a	n/a
US-19	Loyalty points, subscription meal plans, group orders, scheduled orders.	Won't (future)	n/a	n/a

9. Top 5 risks

ID	Risk	Trigger	Likelihood / impact	Owner	Mitigation
R-01	CRUD or scope coverage is rejected, so unbuilt classes must be built	Lecturer rejects the CRUD mapping (5.1)	Medium / High	Dat	Ask L1 at the presentation; decide by 31/10. Extra classes reuse the UC-06 admin pattern at about 6 person-hours each, built by Quan in I3 and I4 from the I3 and I4 buffers; the I5 buffer stays untouched (scenarios in 5.1)

ID	Risk	Trigger	Likelihood / impact	Owner	Mitigation
R-02	Business rules are unconfirmed or change after coding	Client answers differ from the proposed defaults	High / Medium	Dat	Keep every proposed rule in the Assumption Log; keep thresholds and fees in configuration; trace each answer to the tests it changes
R-03	Peak-hour capacity race: two orders are accepted past capacity, or order states drift	Boundary test or concurrent-order test fails	Medium / High	Dat	Count active orders and insert the order in one transaction (SQLite `BEGIN IMMEDIATE`); boundary test at capacity minus 1, capacity, and two simultaneous orders
R-04	Integration drift between six use cases: different IDs, state names or separate data	A use case needs its own copy of an order or a state not on the fixed list	Medium / High	Dat (integration lead)	One shared schema and seed from I1, kept by Quan L.; one Order flows through every use case; state names frozen after I1; Dat runs the demo scenario at the end of I3 and I4; cross-review every iteration
R-05	A member is unavailable, or the new member needs longer to start	A day's planned evidence is missing at the daily check, or UC-05 is not moving by mid-I2	Medium / High	Dat	Dat pairs with Quan in I2; every member can run the repository (Docker or venv); cross-review so each knows another member's use case; the I5 buffer day (13/11) and the freeze (12/11); drop Should and Could items first

10. Assumption Log and open items

10.1 Assumption Log (first entries)

ID	Assumption	Reason	Affected work	Owner	Status / confirmed by + date
A-01	One order is cooked by one hub.	Keeps responsibility for the order clear.	UC-01, domain model	Dat	Proposed; team default, open to client correction
A-02	Hub workload = orders in PLACED, CONFIRMED and PREPARING. A hub is busy when this count is at or above its `max_active_orders`. The check runs once, when the order is created.	The case gives no figure; one definition is used by checkout and by the workload report.	UC-01, UC-03, UC-04	Dat	Proposed; awaiting client (Q1)
A-03	The quiet-hub incentive is one configurable delivery-fee discount for a hub under the quiet threshold; threshold and amount stay open and sit in configuration.	The case says the figures will come from the client.	UC-01	Dat	Proposed; awaiting client (Q1)
A-04	Distance is the straight-line (Haversine) distance from seeded coordinates. The maximum radius and the fee formula are open.	No map service is used; the formula is the client's decision.	UC-01	Dat	Proposed; awaiting client (Q2)

ID	Assumption	Reason	Affected work	Owner	Status / confirmed by + date
A-05	Payment is COD for everyone plus an online team-built stub. Local customers get e-wallet or domestic bank QR; tourists get international card.	The case says the options differ but not what they are.	UC-01	Dat	Proposed; awaiting client (Q4) and lecturer (L2)
A-06	Payment outcomes: online payment fails, no order is created. The kitchen rejects, the customer cancels or delivery fails: a COD payment is VOIDED and an online payment becomes REFUND_PENDING and is owed back in full, with no decision. Only a complaint goes to office staff; a denied complaint leaves the payment PAID and the customer is told why.	Every money path needs a defined end, even though the payout is not built in this release.	UC-01, UC-02, UC-05	Dat	Proposed; awaiting client (Q3)
A-07	Report definitions: sales are DELIVERED orders by delivered date; workload is a snapshot of active load against capacity; waiting for pickup is READY to PICKED_UP and delivery time is PICKED_UP to DELIVERED; failure rate is failed over delivered plus failed.	Expected values are calculated from these definitions before coding.	UC-04	Quan L.	Proposed; awaiting client (Q5)
A-08	The supporting use cases in Section 5, including cancel or edit order (the Cancel order activity, Figure 1e, grey), are analysed but not built; CRUD coverage of their classes is by analysis and seed data.	The build is limited to the six main use cases and the minimum support.	Sections 3 and 5.1, Part 2 CRUD verification	All	Proposed; awaiting lecturer (L1)
A-09	Complaint and refund (including executing a refund) and rating are deferred core features: they stay in the process (grey in Figure 1), the use case list and the backlog, but are not built in this release, so online payments wait in REFUND_PENDING.	Keeps the scope to what can be analysed, built and tested in six iterations.	Section 3, scope	All	Proposed; awaiting lecturer (L3)
A-10	Part 1 is presented live.	The lecturer approved a live presentation in place of the recorded video (verbal approval).	Presentation	All	Confirmed verbally by the lecturer (before 19/10/2026)
A-11	The team has three members from Iteration 2: Quan joins with UC-05 and UC-06, so the team owns six main use cases. Iteration 1 was planned and run by two.	Section 2.1 of the requirements: a team of three owns six main use cases.	Sections 4, 5, 6	Dat	Proposed; awaiting lecturer (L5)

10.2 Open items

- Source repository: https://github.com/Dat071104/SaigonBite (tag `i1-evidence`).
- Effort: about 4 focused hours per student per day is an assumption (Section 6.2), re-checked at the end of every iteration.
- Contribution: every member commits their own work under their own name; the Contribution Statement is signed by all three for Part 2 (I5).

- Client Q&A: Iteration 1 is built on the assumptions above. Each client answer updates the Assumption Log and the tests it affects in the next iteration.

11. Questions

11.1 Client questions

Each question asks for one decision. Where a second point depends on it, the team states the default it will use if no answer comes.

#	Question (one decision)	Proposed default
Q1	What is the workload policy: when is a hub too busy to take orders, and when does a quiet hub get a fee discount, and how much?	Busy when active orders (PLACED, CONFIRMED, PREPARING) reach the hub's capacity. Quiet when active orders are below a quarter of capacity: 5,000 VND off the delivery fee. Both figures sit in configuration.
Q2	How should the maximum delivery distance and the fee be calculated?	Straight-line kilometres from the hub; fee is a base fee plus a rate per kilometre; the team needs the maximum distance and the rates.
Q3	Until when may a customer cancel, and how is an online payment refunded?	Cancel while the order is PLACED or CONFIRMED, not once it is PREPARING. After a rejection, a cancellation or a failed delivery, COD is voided and an online payment is refunded in full with no decision; only complaints are decided by office staff.
Q4	Which online payment options should local customers and tourists each have?	Local: e-wallet or domestic bank QR. Tourist: international card. COD counts as paid when the shipper confirms collection.
Q5	Should sales reports use the delivered date or the placed date?	Delivered date. Delivery time is reported as two intervals, READY to PICKED_UP and PICKED_UP to DELIVERED.

11.2 Questions for the lecturer

The team asks these at the Part 1 presentation and needs the answers by 31/10, the end of I2, so that any extra build fits before the Part 2 report.

#	Question	Related
L1	Is analysis plus seed data acceptable for the CRUD coverage of the unbuilt supporting use cases (mapping in 5.1)? If not, which classes must be built?	A-08, R-01
L2	Is a team-built payment stub acceptable for online payment?	A-05
L3	May complaint and refund and rating be deferred from the build while staying in the process and use case list?	A-09
L4	Our team chose Python 3.13 with Flask; the report template gives examples in other languages. Please confirm Python is acceptable.	Section 2.2
L5	Please confirm the third member and the two use cases assigned to the third member (UC-05 Confirm or reject an incoming order, UC-06 Manage brands and menu items).	A-11

Appendix A. Generative AI Usage Log

The team keeps the full prompts and outputs and will hand them over within 48 hours if asked.

Tool and date	Purpose	Prompt (summary)	Kept or changed	How verified
ChatGPT ("Sol"), 13/10/2026	Initial analysis of the case and the requirements	Asked for candidate classes, events, use cases and open questions from the case study and requirements	Used as a reading aid and a checklist of ambiguities; the team chose its own four use cases	Compared with the case study text and the requirements document
Claude, 15/10/2026	Team plan v1 and v2	Asked for scope, use case split, calendar, CRUD mapping, payment and report rules for two students	Plan v2 was the team's working plan; use case split, scope and calendar were decided by the team	Checked against the requirements, rubric and case; decisions recorded in the plan
ChatGPT, 15/10/2026	Part 1 draft package and an audit of it	Asked for a proposal, slides and speaker scripts, then for an audit against the rubric	Problem-to-capability table and Iteration 1 acceptance criteria idea reused; the rest replaced	Reviewed against the requirements and marking sheet
Claude, 19/10/2026	Strict review of the 15/10 package	Asked to find errors, meta text, undecided choices and rubric gaps	Findings used as input to the 19/10 rebuild	Each finding checked against the package and the requirements
Claude (Opus and Sonnet subagents), 19/10/2026	Rebuild of the proposal, slides, process diagram and plan v3 from the team plan, then a consistency and rubric check	Gave the team plan, case, requirements and a brief with fixed facts; asked for proposal text, slide content, the swimlane diagram and a check against the rubric	Proposal text, slides and Figure 1 generated from the team plan and brief; every member must be able to explain the parts they present. Team review on 20/10/2026; main edits: refund and cancellation rules made consistent across process, state machine and code; scope labelled as planned versus built; actor and risk wording corrected; team of three added	Checked against the requirements, the case and the brief; two timed rehearsals with all three presenters: planned for 24/10 and 25/10
Claude (Sonnet subagent), 19/10/2026	Iteration 1 code in the local repository	Asked for the code skeleton, schema and seed, the UC-01 subset and a pytest suite for the acceptance criteria AC-1 to AC-9	Generated the Iteration 1 code skeleton, schema and seed, UC-01 subset and pytest suite in the local repository; the team reviewed and changed: Dat read the controller, DAOs and tests, using Claude and Codex as review checklists; two defects reproduced on a temporary database and fixed with tests (a6ccbee), cancellation rule aligned with the proposal, UC-01 activity diagram added (40ef14e); a second review closed a request that skipped the price check (e93336a)	pytest run and fresh-clone run recorded in Section 7.3; team review on 20/10/2026
Claude and Codex (OpenAI), 20/10/2026	Independent review of the Part 1 package and the Iteration 1 code before submission	Asked each tool to find logic errors, inconsistencies between proposal, diagram, state machine and code, and claims stronger than the evidence	Accepted: refund and cancellation rules made consistent; price-change and archived-hub defects fixed with tests; scope labels and slide wording corrected; team of three added. Rejected: changing risk owners or swimlanes only for balance	Each finding reproduced on a temporary database or checked against the requirements before it was changed; full test suite re-run on a fresh clone